



GM

Sui workshop · APU × BCC · 40-minute core

Raphael / Raph

Mysten Labs · Solutions Engineer · See you on campus in a few days

Why this call

Hackathon day one should be building—not decoding
blockchain terminology.

What we'll cover

First, Web3

True on any blockchain. Nothing here is Sui-specific.

Keys and addresses

The secret that signs, and the name you hand out.

Transactions and signatures

The request, and the proof that you wrote it.

Wallets

Reviewing, signing, and keeping the account recoverable.

Contracts and gas

Code that runs on a chain, and what it costs to run.

Then, Sui

Where Sui stops looking like everything else.

Objects and ownership

Owned, shared, or immutable — who may use a thing, and who has to wait.

Move and capabilities

Abilities, objects inside objects, and holding an object that **is** permission.

The stack, then three apps

Display, Walrus, Seal, DeepBook — and what gets built out of them.

The web you know

■ Your login

You sign in with an email and password.

■ The company

Its server stores your account and decides which changes are allowed.

Web3 changes who approves

You approve a request. The blockchain checks that approval before making the change.

Private key

A secret number used to sign transactions. Anyone who gets it can sign as you. Never share it.

Real world

■ House key

Whoever holds it can open the door. You do not post a photo of the key.

■ Bank PIN

The bank never asks you to type it into a stranger's form. Anyone who has it can empty the account.

Same job as that PIN

Twelve words, or one long number. Fake data — do not import it.

1 maple

2 river

3 lantern

4 copper

5 quiet

6 orchard

7 silver

8 meadow

9 anchor

10 pebble

11 canyon

12 velvet

`0x3a1f9c8e2b74d0a1...b7e2`

32 bytes

Public Key

Your account is identified by a public address/public key. Your public key is derived from your private key.

Real world

■ Account number

Printed on every statement. Safe to give a friend so they can send you money.

vs

■ PIN

Stays in your head. Sharing the account number does not share the PIN.

The number you can share

Account 1

Address

0x7a1B...C4d2

Balance0.42

Transaction

A request to change something on the blockchain—for example, transfer a ticket to another account.

Real world

The cheque is the transaction. If the bank takes it, it tries to execute this.

Jordan Lee 12 Jalan Ampang, Kuala Lumpur	001 DATE · DD MM YYYY <i>30 / 08 / 2026</i>
PAY TO THE ORDER OF <i>Alex Tan</i>	RM 50.00
<i>Fifty ringgit and --- 00/100</i>	RINGGIT
Riverside Bank, 235 Main Street, Kuala Lumpur	
MEMO <i>Tickets</i>	SIGNATURE
001 0001924 00050	

Same cheque in a wallet

MetaMask

Confirm

Send

0.01 ETH

From

0x7a1B...C4d2

To

0x3f91...88ae

Fee

~\$1.20

Reject

Confirm

What the network receives

```
from 0x7a1B...C4d2
to    0x3f91...88ae
value 10000000000000000
nonce 17
gas    21000
chainId 1
data 0x
```

Signature

Created with the private key for one exact transaction. It proves that transaction was signed with the private key, without revealing it.

Real world

Same cheque — now signed. The bank checks it is not tampered with, then executes.

Jordan Lee 12 Jalan Ampang, Kuala Lumpur		001
		DATE · DD MM YYYY <i>30 / 08 / 2026</i>
PAY TO THE ORDER OF <i>Alex Tan</i>		RM 50.00
<i>Fifty ringgit and --- 00/100</i>		RINGGIT
Riverside Bank, 235 Main Street, Kuala Lumpur		
MEMO <i>Tickets</i>	<i>Jordan Lee</i> SIGNATURE	
001 0001924 00050		

Signing a tx

```
hash      = Hash(transaction)  
→ 0x9c2a...f41b
```

```
signature = Sign(hash, private_key)  
→ 0x5e8d...c01a
```

From cheque to proof

■ 1 · Cheque

Send 0.01 to `0x3f91...88ae`.

■ 2 · Hash / Sign

`Hash(tx)` → `0x9c2a...f41b`

`Sign(hash, private_key)` → `0x5e8d...
c01a`

■ 3 · Check

Not tampered. Matches the account. Then it runs.

Wallet

An app that manages your account and helps you use it. The assets remain on the blockchain.

Create

Build the transaction you requested.

Review

Show what the transaction will change.

Sign and send

Use the private key to sign. Some wallets hide this step behind social login.

Wallets you may recognize



MetaMask

Commonly used with Ethereum apps.



Trust Wallet

A multi-chain mobile wallet.



Phantom

Commonly used with Solana apps.



Slush

The official Sui wallet by Mysten Labs.

Smart contract

■ Real world

A vending machine. You press one button. Its rules decide whether it drops a can.



■ On chain

`to` = the machine. `data` = `buyTicket(42)`. Code accepts or rejects.

Fees (Gas in ethereum)

You pay fees depending on how much computing resources you use.
Ethereum calls it gas fees, some chains just call it Fees.

Gas is calculated differently

■ Ethereum

The processing charge — like RM 0.10 on a RM 50 transfer — is price × work. 21,000 gas × gwei, paid in ETH.

vs

■ Sui

A different formula: work plus storage, minus a rebate if storage is freed. Paid in SUI. The smallest unit is MIST.

Sui // Objects

Keys, accounts, and signatures still apply. Data lives as objects.

Sui foundations

■ Object

■ Ownership

■ Paths

■ Capabilities

Object

A thing on the chain with its own ID. A transaction can use it or change it.

Real world

A concert ticket. One copy. In your hand. Hand it over, you no longer have it.



Same ticket on Sui

```
id      0x8f2a...c19d
type    workshop::Ticket
owner   0x7a1B...C4d2
version 3
```

```
event   APU Night
seat    B12
```

Ownership

Who is allowed to use a thing on the chain — and whether anyone else has to wait their turn.

Two questions

■ Who may use it

Some things should belong to one person. Others are meant for a whole group to change.

■ Does anyone wait

If a thing is yours alone, nobody is queued behind you. If a group shares one thing, they take turns changing it.

Three kinds of ownership

■ Owned

Who one account — yours.

Waiting nobody waits on you.

■ Shared

Who anyone. The app decides who may change it.

Waiting people changing it at once take turns.

■ Immutable

Who anyone can read it. Nobody can change it.

Waiting nothing to take turns over.

What waiting looks like

Time runs left to right. Dots are time spent waiting.

Their own ticket each

nobody waits

```
Amir    ticket #1    --> done
Bea     ticket #2    --> done
Chin    ticket #3    --> done
```

The same water dispenser

they take turns

```
Amir    dispenser    --> done
Bea     dispenser    ....--> done
Chin    dispenser    .....--> done
```

Mysticeti v2

Sui's consensus. A DAG: every validator proposes at once, and there is no separate round to certify blocks. On mainnet since July 2024, and v2 is the default from node v1.60.

What Mysticeti does

It agrees the order of transactions that cannot skip ordering.

Shared writes

The commit order is the write order on each shared object.

Parallel cores

Different shared objects still execute on different cores after that order is known.

Not every transaction

A simple owned transfer never waits on this path.

How it got here

■ Narwhal + Bullshark

Spreading transactions and ordering them were two layers. Every block was certified before it could commit.

■ Mysticeti v1

The certifying round is gone. Fewer rounds, lower latency, same safety story. Mainnet July 2024.

■ Mysticeti v2

Checking whether a transaction is allowed moves inside consensus. Same commit rule as v1.

Owned: your APUBCC workshop ticket

You carry it. You can hand it to a friend. You can spend it.
A stranger cannot take it, and cannot hand it over for you,
and cannot spend it.

Ticket

Owned

Use it **you**

Change it **only you**

Waiting **nobody waits on you**

Owned on Sui

One account holds the ticket. Handing it over is one function call.

```
public struct Ticket has key, store {  
    id: UID,  
    event: String,  
    seat: String,  
}  
  
public fun give(ticket: Ticket, to: address) {  
    transfer::public_transfer(ticket, to)  
}
```


Giving it away changes the owner

■ Before

The ticket belongs to your account.



■ After

You sign one transaction. Your friend's account holds it. You do not.

Shared: the water dispenser

Anyone in the building can use it. There is one spout, so the second person waits until the first has filled their cup.

Dispenser

Shared

Use it

anyone

Change it

whoever the app allows

Waiting

they take turns

Shared on Sui

Nobody owns the dispenser. Everyone calls the same function on the same object.

```
public struct Dispenser has key {
    id: UID,
    cups_left: u64,
    paused: bool,
}

fun init(ctx: &mut TxContext) {
    transfer::share_object(Dispenser {
        id: object::new(ctx),
        cups_left: 500,
        paused: false,
    })
}

public fun dispense(d: &mut Dispenser) {
    d.cups_left = d.cups_left - 1;
}
```

Immutable: the printed programme

Anyone can read it. Nobody can edit that copy — not even the person who printed it. The code you publish works the same way.

Programme

Immutable

Use it

anyone can read it

Change it

nobody

Waiting

nothing to wait for

Immutable on Sui

Freezing is one call, and there is no way back.

```
public struct Programme has key {  
    id: UID,  
    event: String,  
    lineup: vector<String>,  
}  
  
public fun publish(programme: Programme) {  
    transfer::freeze_object(programme)  
}
```

Two ways a tx runs

■ Nobody waits

Your own things, and things nobody can change. Someone else's ticket never waits on yours.

vs

■ They take turns

Anything a group can change. Turns are taken on that one object only.

Flash quiz

Three questions on objects and ownership. Shout out the letter.

QUESTION 1

You are selling 500 tickets to one event. One ticket per attendee. What should each ticket be?

A Owned

B Shared

C Immutable

ANSWER 1

You are selling 500 tickets to one event. One ticket per attendee.

A Owned

B Shared

C Immutable

Owned. Each attendee holds their own ticket, so nobody queues behind anybody. Shared would put 500 people in one line for no reason. Immutable would mean the ticket can never be scanned or resold.

QUESTION 2

Two people buy from the same shared shop object in the same second. What happens?

- A** One of them fails and has to try again
- B** Both go through, one after the other
- C** Both fail, and the shop is stuck

ANSWER 2

Two people buy from the same shared shop object in the same second.

A

One of them fails and has to try again

B

Both go through, one after the other

C

Both fail, and the shop is stuck

Taking turns is not failing. Consensus picks an order and both run. And the queue is only for that one object — every other shop on Sui is unaffected.

QUESTION 3

**You froze the programme object, then spotted a typo in it.
How do you fix the typo?**

A Call a setter with the AdminCap

B Unfreeze it, edit, freeze again

C You cannot — publish a new one

ANSWER 3

You froze the programme object, then spotted a typo in it.

A Call a setter with the AdminCap

B Unfreeze it, edit, freeze again

C You cannot — publish a new one

Freezing has no undo. Not for you, not for an admin, not for the person who published it. Point people at the new object instead — which is why you freeze late, not early.

Move

The language for Sui contracts. Objects are first-class. Abilities decide what the type may do.

Four abilities

■ key

May live in global storage.
Needed for objects.

■ store

May live inside another object.
Needed to transfer freely.

■ copy

May be duplicated. Coins and
tickets must not have this.

■ drop

May be discarded. Valuable
objects must not have this.

An object inside an object

■ Real world

A ticket sealed in an envelope. You reach the ticket by opening the envelope.



■ On chain

`store` lets one object sit in another object's field. It is not a fourth kind of ownership — the envelope is what you own.

Capability

An object that is permission. Holding it is the right to call the function.

Real world

A staff key to turn off the water dispenser. Water dispenser does not care who is trying to turn it off. It just cares if you have the key.

DispenserAdminCap

Owned

Type school::DispenserAdminCap

Owner 0x7a1B...C4d2

Job pause the dispenser

Same idea on Sui

`pause` never asks who you are. It asks for a reference to the key.

The abilities on the struct decide who is allowed to move that key.

```
public struct AdminCap has key {  
    id: UID,  
}  
  
public fun pause(  
    d: &mut Dispenser,  
    _: &AdminCap,  
) {  
    d.paused = true  
}
```

Who can move the AdminCap

- **key** only

Only your module can move it. Handover still happens — through a transfer function you write, with the rules you choose.

vs

- **key + store**

The holder can send it anywhere, or nest it inside another object and share that object. Add **store** only when the key must live inside something else.

Which capability

■ AdminCap

Pause, fees, config on a shared protocol.

■ OwnerCap

This kiosk, this vault. Bind the cap to that object's ID.

■ Publisher

Display and transfer rules for a type you published.

■ Witness

Once, at publish. Proves "this module, this moment."

You do not need a sender check

Before your function body runs, the network has already checked that every owned object in the transaction belongs to the signer. Nobody can pass in an `AdminCap` they do not own.

So `&AdminCap` in the signature *is* the permission check. Comparing `sender()` next to it protects nothing.

A `sender()` check only earns its place when no owned object gates the call — an admin address stored inside a shared object, for example.

```
public fun pause(  
    d: &mut Dispenser,  
    _: &AdminCap,  
    ctx: &TxContext,  
) {  
    // redundant code  
    assert!(  
        ctx.sender() == d.admin,  
        ENotAdmin,  
    );  
  
    d.paused = true  
}
```

The `assert!` is dead weight. The caller could not have supplied `&AdminCap` without already owning it.

Three safety rules

■ Keep secrets private

Never share a private key or recovery phrase.

■ Check before signing

Open slush.app and read what the wallet asks you to approve.

■ Use Testnet first

Practice with a test account and test tokens before Mainnet.

What makes Sui Move different

■ Objects, not accounts

Everything an app tracks has its own ID and its own owner. There is no single contract balance sheet.

■ Ownership sets the queue

Owned, shared, or immutable. The kind you pick decides who has to wait.

■ Parallel by default

Transactions touching different objects run together. A simple owned transfer skips consensus.

■ Abilities

`key`, `store`, `copy`, `drop`. The type itself declares what may happen to it.

■ Prevent copying/deleting Asset

No `copy`, no `drop` — so the compiler refuses to duplicate a coin or quietly throw one away.

■ Capabilities, not lists

Hold the object, hold the right. The network checks that before your code runs.

■ Nothing can re-enter

Every call is known when you compile. No unknown code takes over mid-function, so re-entrancy has no analogue here.

■ Deleting pays you back

Free the storage and 99% of what you paid comes back. Frozen objects never can — they lock it forever.

On the stack

■ Display

A template so wallets and explorers show a name, an image, and fields — not a raw type.

■ Seal

Encrypt a file; a Sui policy decides who may decrypt it. Often paired with Walrus.

■ zkLogin + Passkey

Sign in with an account or a device you already have. No phrase to write down.

■ Derived objects

A child ID from a parent + a key. Registries without one shared bottleneck.

■ Nautilus

Heavy or private work runs off chain in a sealed enclave. It hands back a proof Move checks first.

■ Enoki

Your app sponsors the gas and hosts the login. A first-time user needs no SUI and no extension.

■ Kiosk + TransferPolicy

The shop, and the checklist a sale must pass. Royalties that still apply on resale.

■ DeepBook

A native order book. Other apps send buy and sell orders into it.

■ PTB

Programmable transaction blocks. Chain many calls into one transaction, passing results along. All of it lands, or none does.

■ Walrus

Large files live here, not on the chain. A Sui object points at the blob.

■ SuiNS

A `.sui` name instead of a long address — for accounts and for sites.

■ Clock + Random

Time and randomness the network hands you. No oracle, no off-chain dice to trust.

Example 1 · A digital shop

A marketplace for unique items. You list an object; someone buys it; the creator's rules still apply on resale.

Display

The item is an object you own. Display is why the wallet shows a picture and a name—not a raw type.

Capability

A publisher/admin cap lets the creator set Display and trading rules. No cap, no admin.

Kiosk + TransferPolicy

Kiosk is the shop. TransferPolicy is the sale checklist (royalty, allowlist). A sale leaves a hot-potato request that must be resolved in the same transaction.

Example 2 · An on-chain exchange

DeepBook is a shared order book. Apps plug in. Matching happens on Sui, not on a company's matching engine.

Shared object

The book is one object many traders touch. Those trades **do** wait on each other—that is the honest tradeoff.

Parallel work

Unrelated markets, and wallets moving **owned** coins, can still run beside it. Contention is only where objects overlap.

Coins + PTBs

You trade **Coin** objects. One programmable transaction can swap, then send the result onward. Each app does not ship its own exchange.

Example 3 · A website on Walrus

HTML, images, and scripts are blobs on Walrus. A Sui object says who owns the site. The name can be a `.sui` name.

Walrus + Walrus Site

Files are too big for the chain. Walrus stores them; Sui stores the pointer and the owner.

SuiNS

The site is an object you can transfer. SuiNS is the human name—like a domain, held as an object.

Seal (optional lock)

Encrypt a page or a drop. Only addresses that pass the on-chain policy get the key.

Agentic Engineering

■ 1 · Research

The model reads the code and writes down what is actually there — files, lines, how things flow. It writes no code in this step.

■ 2 · Plan

Those findings become steps you can read: what changes, where, and how you will know it worked. This is the part you correct.

■ 3 · Implement

A fresh session builds from the approved plan. It needs the plan, not the whole conversation.

A wrong plan costs seconds to fix; a wrong implementation costs your afternoon.

The Model gets worse the more you fill its head — so keep each step short.

Give these a go

■ Explore

docs.sui.io

Guides, references, and every piece of the stack we just walked through.

■ Give your agent the docs

docs.sui.io/getting-started/sui-mcp-server

Four lines of config. Your agent reads current Sui docs instead of guessing from memory.

Questions